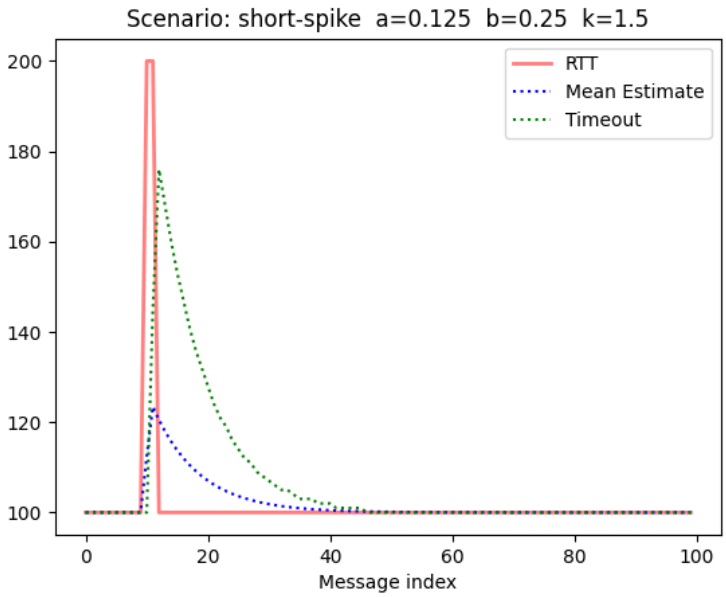
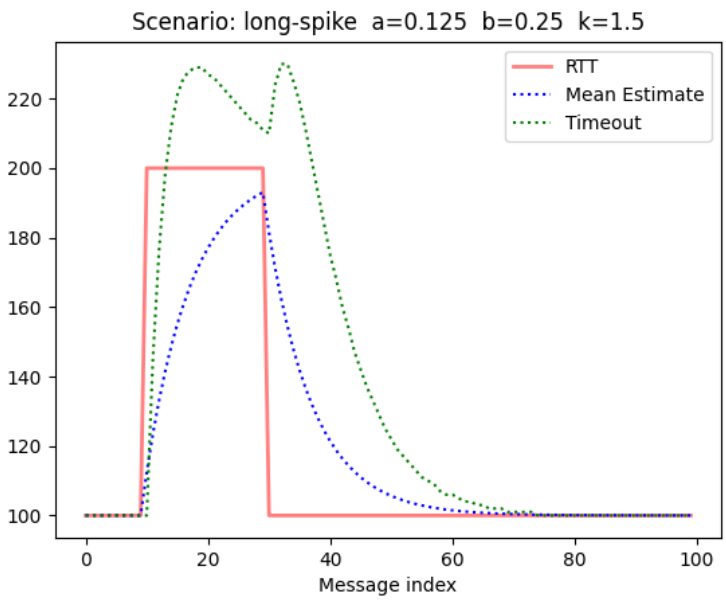
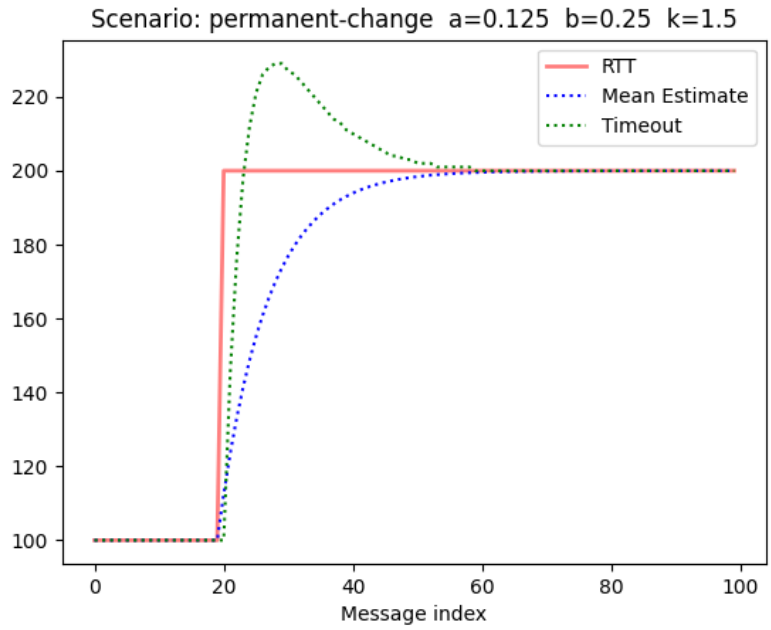
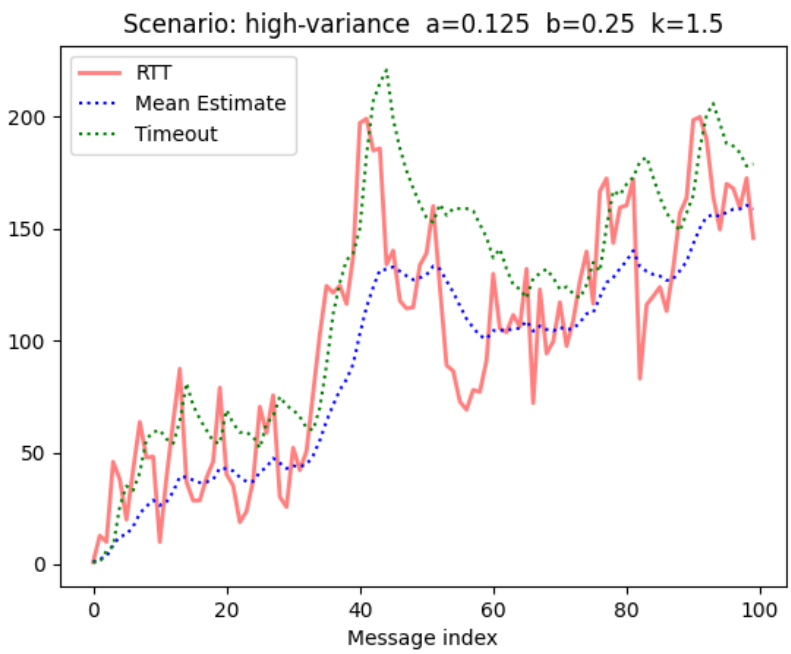
2.3

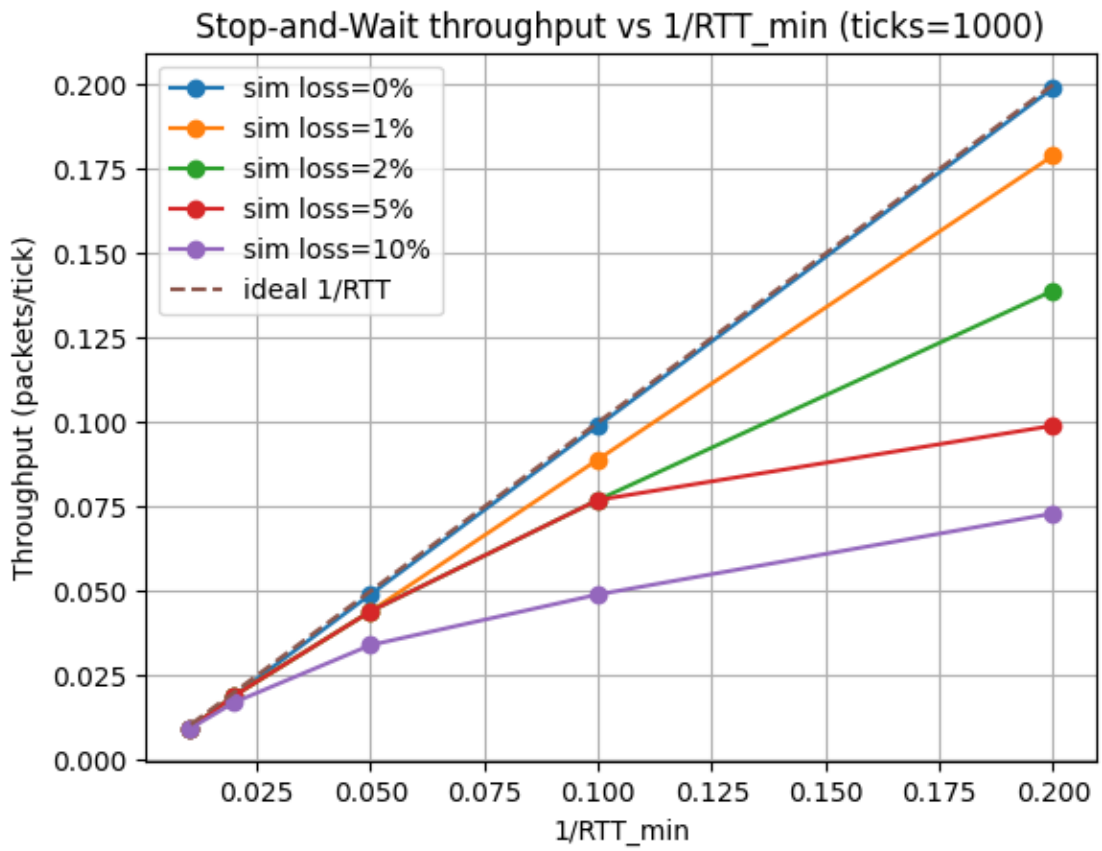

Scenario: short-spike a=0.125 b=0.25 k=1.5
RTT
Mean Estimate
Timeout
200
180
160
140
120
100
0
20
40
60
80
100
Message index


Scenario: long-spike a=0.125 b=0.25 k=1.5
RTT
Mean Estimate
Timeout
220
200
180
160
140
120
100
0
20
40
60
80
100
Message index


Scenario: permanent-change a=0.125 b=0.25 k=1.5
RTT
Mean Estimate
Timeout
220
200
180
160
140
120
100
0
20
40
60
80
100
Message index


Scenario: high-variance a=0.125 b=0.25 k=1.5
RTT
Mean Estimate
Timeout
200
150
100
50
0
0
20
40
60
80
100
Message index

3.3 Stop-and-Wait



(1) Is the throughput close to 1/R?

Yes. The throughput observed in the lossless simulation is very close to 1/R (small deviations are caused by simulation discretization/initial padding). For example, when R=10, the observed throughput is approximately 0.099 $\approx$ 0.1.

(2) Does the protocol still function correctly with a small amount of packet loss ($\approx$1%)?

Yes, it will continue to function correctly, but the throughput will decrease slightly. Theoretically, the expected number of transmissions for each successful packet is approximately 1/(1−p), so the expected throughput is approximately (1−p)/R (approximately 1/R with a small p, a reduction of approximately p×100%). Therefore, packet loss only reduces the available throughput by a constant factor and does not cause the protocol to collapse.

(3) If the simulated throughput is significantly less than 1/R, what is the reason?

When running simulations in a high drop rate (5/10%) scenario, we observe significant decrease in throughput. This is because the period of time out (100ms) is much longer than RTT. Therefore, when drop rate is higher, with the decrease of RTT, the proportion of timeout taking within the entire time period increases dramatically, causing a collapse in.

(4) Regarding divergence with R: increasing or decreasing?

Ideal analysis ($T \approx (1-p)/R$) indicates that the relative drop is approximately $(1-p)$, independent of R—that is, the "relative" deviation does not change significantly with R. In simulations, you may see small differences (e.g., the relative drop appears more pronounced with small R), due to small-sample effects caused by discretization, initial padding, and the timing of timeouts and retransmissions. Overall conclusion: Stop-and-Wait results in a relatively constant throughput loss at 1% packet loss, which does not systematically increase with RTT.

3.4 Sliding Windows

(1) Is the throughput close to W/R?

When the window can fill the channel ($W \gtrsim$ bandwidth-delay product) and packet loss is minimal with reasonable timeouts, the observed throughput is close to W/R when RTT is little. With the increase of RTT, however, the throughput of the network with large W reaches a bottleneck and does not increase anymore. This is because at very low RTT, signal delay has a negligible impact on transmission rate; the upper limit of network bandwidth restricts the transmission rate. On the other hand, if W is very small (e.g., W=1), it degenerates into Stop-and-Wait.

(2) Does the protocol still work correctly with a small amount of packet loss ($\approx 1\%$)?

It will continue to work, but the throughput impact pattern is more complex than Stop-and-Wait: packet loss causes some packets within the window to be retransmitted, occupying retransmission time slots. If the timeout/retransmission strategy causes the window to be "stuck," there will be a significant decrease in throughput. However, when W is large, a packet loss only affects a portion of the packets in the window, and the overall throughput decrease is small (a larger W can mask a small amount of packet loss).

(3)If the simulated throughput is significantly less than W/R, what is the reason?

In the sliding-window experiment, we have observed significant margin between W/R and actual performance.

Main reasons:

1. Idle bandwidth due to timeouts and retransmissions (sender waits for a timeout before retransmitting; if the timeout is much longer than the RTT, it creates window gaps).
2. Packet loss results in unfilled slots in the window due to unacknowledged packets (especially in implementations where the sender relies on timeouts rather than fast retransmissions).
3. W is less than the bandwidth-delay product, failing to fully utilize link capacity.
4. Implementation details of ACK arrival and window sliding (e.g., ACK processing timing) also have an impact.

(4)Regarding divergence with R and W:

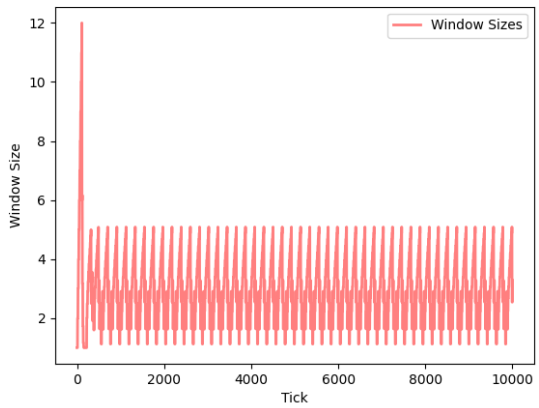
For a fixed W: As R increases, a single retransmission or pause due to packet loss consumes more absolute time, thus increasing the absolute loss in throughput (i.e., the relative deviation often increases, especially when W cannot cover the gaps caused by packet loss). Therefore, "with a fixed, relatively small W, as the RTT increases, the gap between simulation and theory usually widens." For a fixed R: Increasing W will raise the theoretically achievable throughput limit W/R, while also better masking minor packet losses (because even if one packet is lost, other packets in the window are still being transmitted). Therefore, "increasing W will reduce the gap between simulation and theory (especially at low to moderate packet loss rates)."

In summary, the gap under Sliding-Window is a function of W and R: the gap tends to increase as R increases (when W is not large enough); the gap tends to decrease as W increases (closer to W/R).

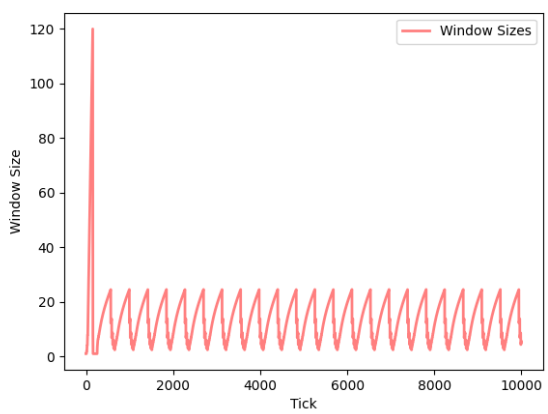
AIMD Summary

From the generated plots, we can see clear sawtooth cycles in the window traces (peaks followed by sharp drops).

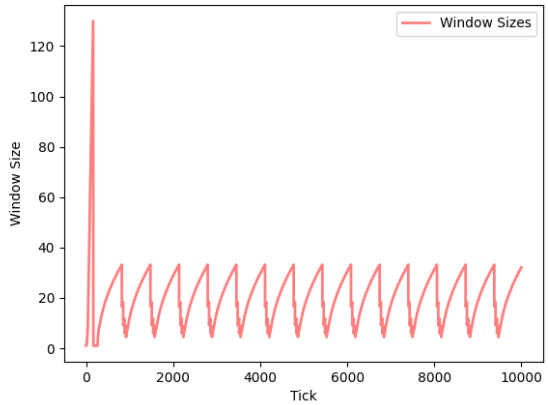
q=1: Median period = 40 ticks ≈ 4.0 RTTs • Max in-order = 854 → Throughput ≈ 0.854 pkt/RTT (ticks=10000, rtt_min=10)



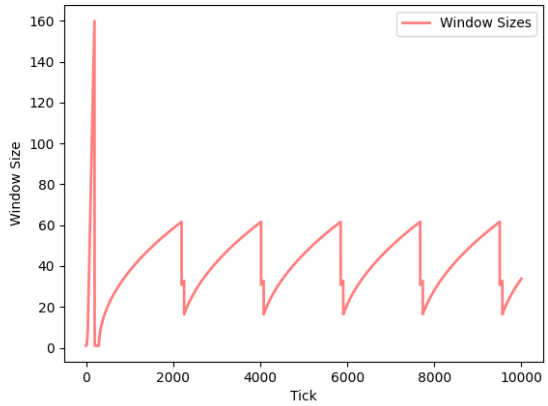
q=10 • Median period = 23 ticks ≈ 2.3 RTTs • Max in-order = 8127 → Throughput ≈ 8.127 pkt/RTT



q=20: • Median period = 32 ticks ≈ 3.2 RTTs • Max in-order = 9400 → Throughput ≈ 9.400 pkt/RTT



q=50: • Median period = 60 ticks ≈ 6.0 RTTs • Max in-order = 9964 → Throughput ≈ 9.964 pkt/RTT



In AIMD , a certain amount of queue capacity is needed to allow the sender to approach the link's full utilization without triggering premature packet loss. Because AIMD increases the sending rate gradually and only reduces it after detecting congestion (typically via packet loss), the sender must slightly exceed the link capacity to discover the available bandwidth. The queue temporarily buffers this excess traffic, preventing immediate packet drops and keeping the link fully utilized while congestion builds up. Without sufficient queue space, packets would be dropped as soon as the sending rate exceeds capacity, causing the congestion window to be cut too early and leading to poor average throughput. Thus, the queue acts as a cushion that absorbs short-term rate overshoot, enabling AIMD to achieve throughput close to the link's capacity.